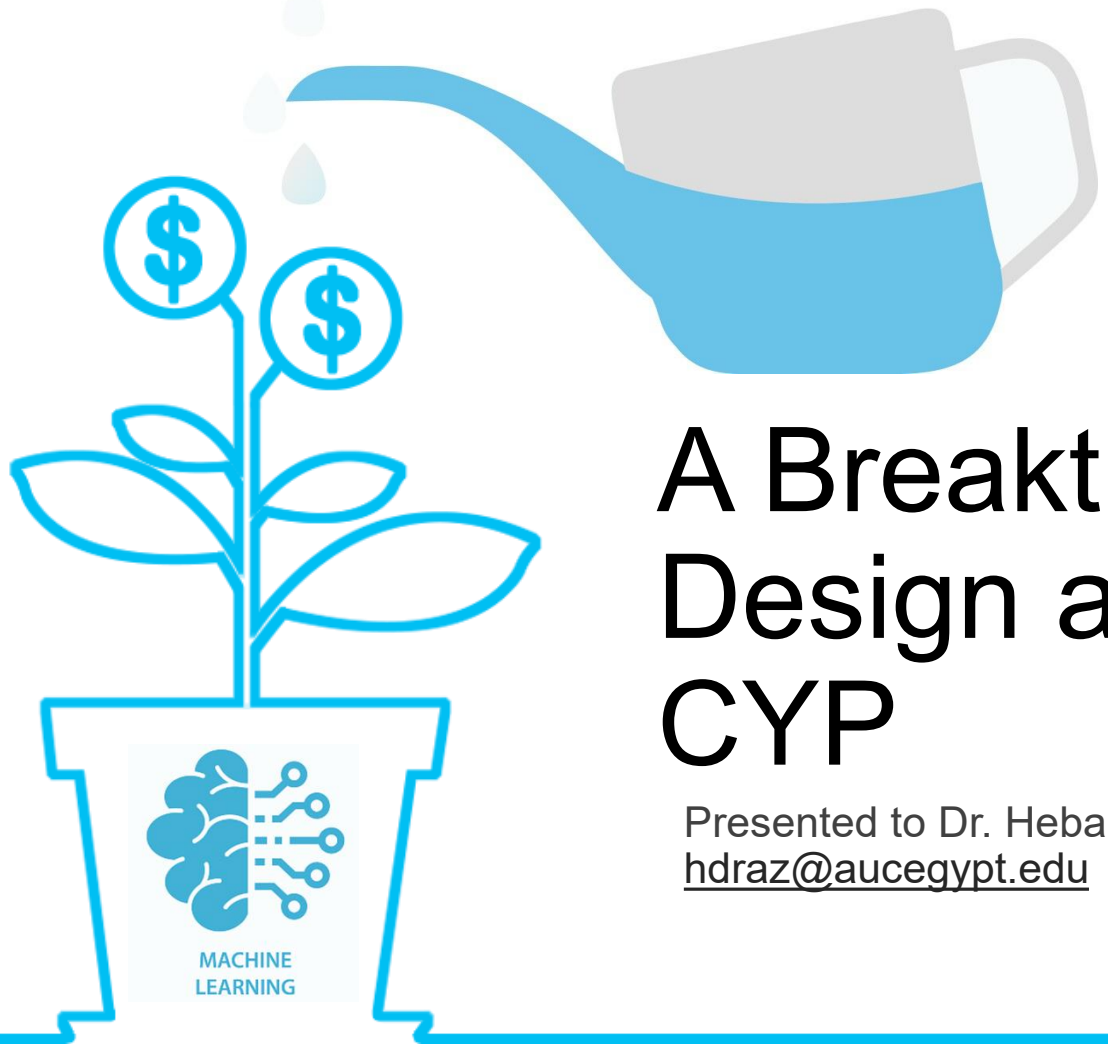
Electronics Research Institute



A Breakthrough in the Digital Design and Verification of the CYP

Presented to Dr. Heba Draz, ECIP Internship Program Committee
hdraz@aucegypt.edu

Course: Hardware/Software Co-Design

Presented by: Ali Adel | ali.adel@email.com

https://eri.sci.eg

Agenda

01 Recap

02 Problems Encountered

03 Solutions

04 New Dataset

05 RTL design

06 UVM Testbench

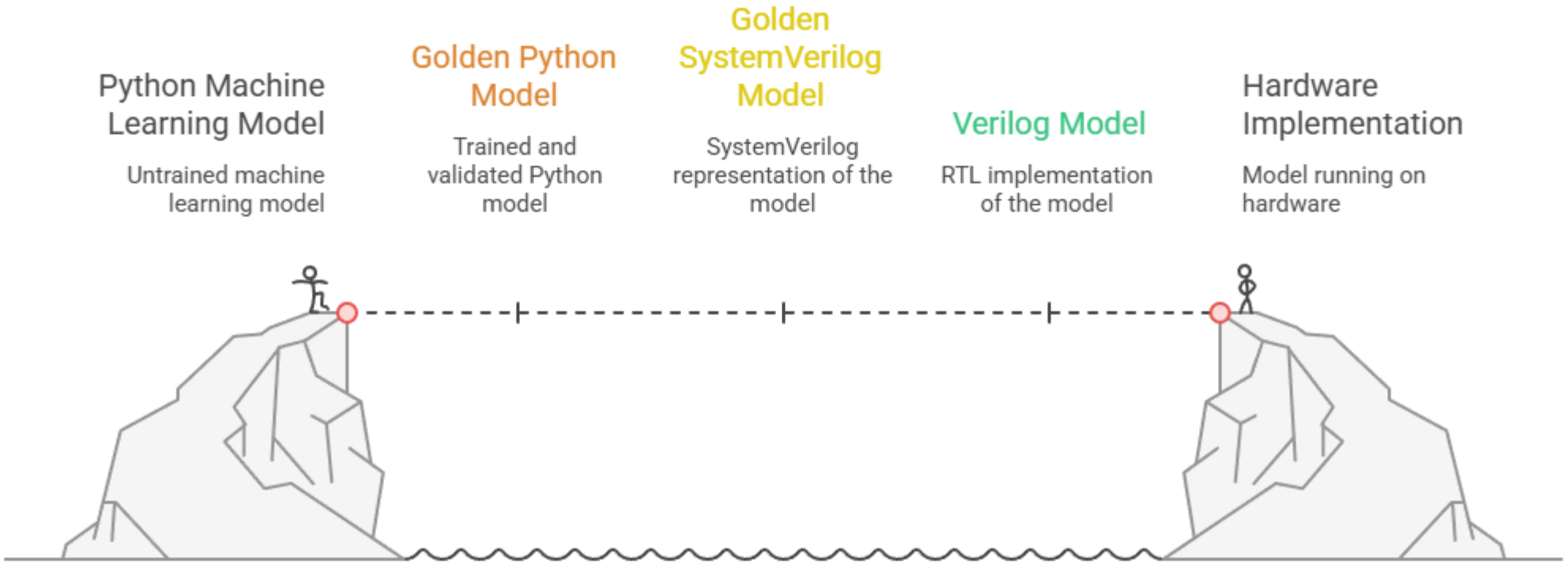
07 Conclusion

Recap

What did we discuss over the past weeks?

Recap

- The model is intentionally non-synthesizable and serves as a functional reference.
- Hardware implementation requires moving away from real types to explicit numeric representations.



Problems Encountered

What exactly did we talk about last week?

Problems Encountered

Encoding non numeric features

datasets > Smart_Farming_Crop_Yield_2024.csv > data

```
farm_id,region,crop_type,soil_moisture_%,soil_pH,temperature_C,
FARM0001,North India,Wheat,35.95,5.99,17.79,75.62,77.03,7.27,No
FARM0002,South USA,Soybean,19.74,7.24,30.18,89.91,61.13,5.67,Sp
FARM0003,South USA,Wheat,29.32,7.16,27.37,265.43,68.87,8.23,Dri
FARM0004,Central USA,Maize,17.33,6.03,33.73,212.01,70.46,5.03,S
FARM0005,Central USA,Cotton,19.37,5.92,33.86,269.09,55.73,7.93,
FARM0006,Central USA,Rice,44.91,5.78,24.87,238.95,83.06,4.92,Sp
FARM0007,North India,Soybean,36.28,7.04,21.8,123.38,47.91,4.02,
FARM0008,East Africa,Maize,27.1,5.72,22.26,296.33,80.34,5.44,Sp
FARM0009,Central USA,Soybean,40.54,6.35,19.24,184.82,76.5,5.21,
FARM0010,East Africa,Rice,10.25,6.92,16.18,66.85,41.57,5.98,Spr
FARM0011,South India,Wheat,13.39,6.36,23.47,166.76,76.45,8.04,D
FARM0012,South USA,Rice,40.61,5.65,31.16,263.99,44.9,7.91,Drip,
```

```
typedef enum int {

WHEAT    = 'h3f800000, //1 IEEE 754 format
SOYBEAN  = 'h40000000, //2 IEEE 754 format
MAIZE    = 'h40400000, //3 IEEE 754 format
COTTON   = 'h40800000, //4 IEEE 754 format
RICE     = 'h40a00000  //5 IEEE 754 format

} crop_type_e;
```

Problems Encountered

Arithmetic Unit Performance Issue

- The arithmetic unit produced incorrect results during verification.
- In some cases, the error percentage exceeded 20%.
- Due to these errors, the decision tree selected incorrect branches.
- This led to inaccurate system decisions, thus no correct yield prediction
- Replacing the arithmetic unit and re-verifying the system is time-consuming.

$$X_{new} = \frac{X_i - X_{mean}}{\text{Standard Deviation}}$$

```
# ------------------------------------------------------------
# | Parameter | Golden Model | RTL Design | % Error
# ------------------------------------------------------------
# | sm        |   0.9389     |   0.9595   |    2.19
# | ph        |   0.8805     |   0.9049   |    2.77
# | temp      |  -0.5377     |  -0.7745   |   44.04
# | rain      |   0.8065     |  -0.4650   |  -42.34
# | hum       |  -1.1804     |  -0.9747   |  -17.42
# | sun       |  -1.7790     |  -1.7152   |   -3.59
# | ndvi      |   0.9029     |   0.9762   |    8.13
# | gd        |   1.5183     |   1.6467   |    8.46
# ------------------------------------------------------------
# ** Note: $stop    : top/top.sv(137)
#    Time: 1995 ns  Iteration: 0  Instance: /tb_top
```

Figure 1: Testbench results showing the accuracy of the scaling equations between the RTL design and the golden model

Solutions

Arithmetic Unit Performance Issue

- Reduce the number of arithmetic operations in both the machine learning model and the RTL design, thus no scaling is applied.
- Change the algorithm of the ‘floating_point_compare_unit’ and verify it.
- The design is currently fully synthesizable!

```
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# ---------------------------------------------------------------------------------------------------------------
# | Row | Actual Yield (kaggle dataset) | Python Golden Model  | Golden Model V3 - With FSM | RTL Design With FSM |
# ---------------------------------------------------------------------------------------------------------------
# |   1 |                          4408 |                 4718 |                       4718 |                4718 |
# ---------------------------------------------------------------------------------------------------------------
# |   2 |                          5390 |                 4376 |                       4376 |                4376 |
# ---------------------------------------------------------------------------------------------------------------
# |   3 |                          2931 |                 4074 |                       4074 |                4074 |
# ---------------------------------------------------------------------------------------------------------------
# |   4 |                          4228 |                 3825 |                       3824 |                3824 |
# ---------------------------------------------------------------------------------------------------------------
# |   5 |                          4980 |                 4012 |                       4012 |                4012 |
# ---------------------------------------------------------------------------------------------------------------
# |   6 |                          4384 |                 4168 |                       4168 |                4168 |
# ---------------------------------------------------------------------------------------------------------------
# |   7 |                          4501 |                 4004 |                       4004 |                4004 |
# ---------------------------------------------------------------------------------------------------------------
# ** Note: $stop    : top/top.sv(131)
#    Time: 3975 ns  Iteration: 1  Instance: /tb_top
# Break in Module tb_top at top/top.sv line 131
# Stopped at top/top.sv line 131
VSIM 2>
```

Figure 2: The testbench output shows that the RTL design passed the initial verification phase successfully

Problems Encountered

Accuracy of the model Issue

- Due to the low quality of the dataset, the machine learning model's accuracy is too low.
- Even after cleaning the dataset and removing outliers.
- As shown in the photo, the maximum accuracy the ML model reached is only 41% with a tolerance of 25%

138

```
Decision Tree: MSE=2506931.051, R2=-0.815, Acc±25%=41.00%
```

Figure 3: Decision tree model accuracy with tolerance 25%: 41%

Solutions

Accuracy of the model Issue

- We can try different models to improve the accuracy.
- Models applied:
  - Random Forest
  - ElasticNet
  - Ridge
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - CatBoost
  - KNN

Solutions

0 → 50%

100%

Accuracy of the model Issue

- We can try different models to improve the accuracy.
- And then choose the top 3 to apply a 15-cycle improvement to them, targeting the lowest MSE and the highest R^2.

```
========== FINAL Model ==========
Random Forest
Lowest MSE: 1395068.736
Highest R2: -0.010
Highest Accuracy : 50.00%
```

Figure 6: Final ML model

```
Decision Tree: MSE=2506931.051, R2=-0.815, Acc±25%=41.00%
KNN: MSE=1617614.366, R2=-0.171, Acc±25%=52.00%
Random Forest: MSE=1417473.556, R2=-0.026, Acc±25%=51.00%
ElasticNet: MSE=1407632.235, R2=-0.019, Acc±25%=44.00%
Ridge: MSE=1442821.014, R2=-0.045, Acc±25%=45.00%
Gradient Boosting: MSE=1756129.776, R2=-0.272, Acc±25%=42.00%
XGBoost: MSE=1811956.427, R2=-0.312, Acc±25%=46.00%
LightGBM: MSE=1559025.491, R2=-0.129, Acc±25%=53.00%
CatBoost: MSE=1507817.933, R2=-0.092, Acc±25%=49.00%
Top 3 selected:
ElasticNet
Random Forest
Ridge
```

Figure 5: Different models' accuracy

➢ What about using a different dataset?

New Approach

What makes the new dataset so special?

New Dataset

Dataset Analysis: Pros

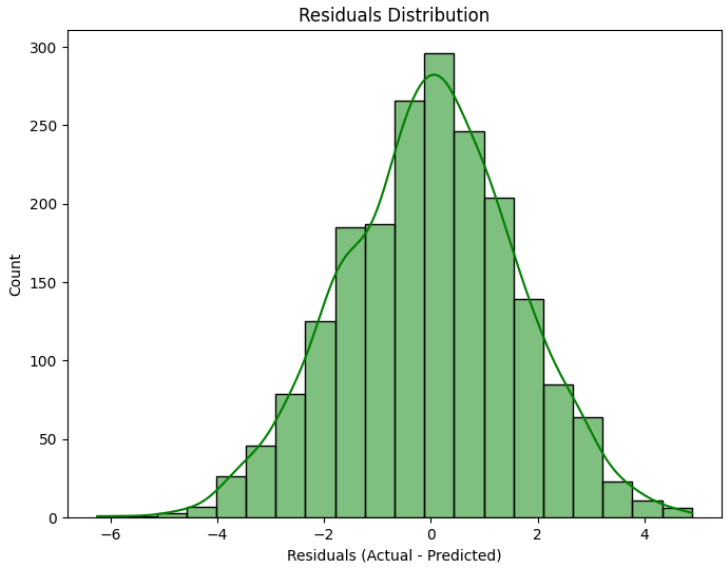


Figure 7: Residuals Distribution

```
Decision Tree: R2: 0.9954, MAE: 1.2964, RMSE: 1.6334
```

Figure 8: Decision tree metrics

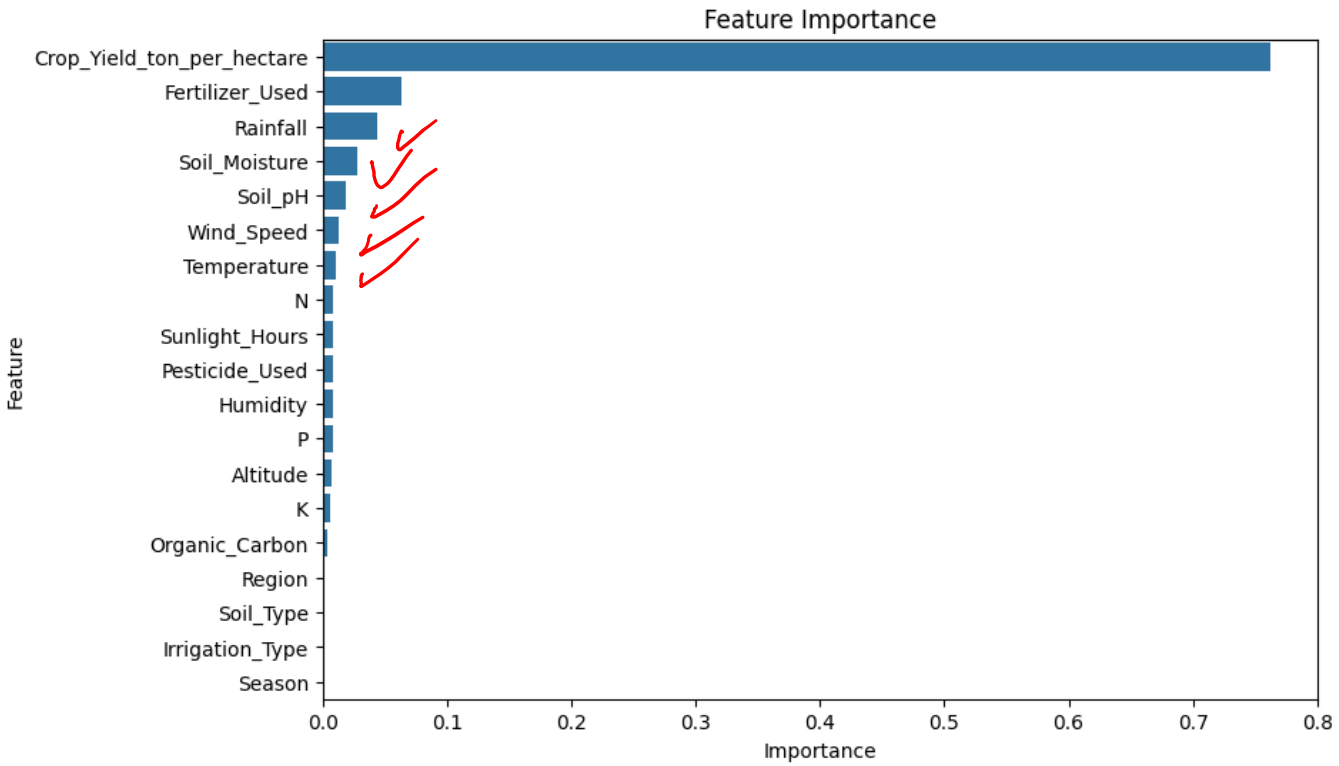


Figure 9: Feature Importance

Reference:https://www.kaggle.com/datasets/miadul/smart-crop-recommendation-dataset/code

New Dataset

Dataset Analysis: Cons

- Poor Generalization:
  - From the Actual vs Predicted Crop Yield plot, it is evident that the data points are clustered in a few narrow yield ranges rather than being evenly distributed across the entire yield spectrum.
  - Specifically, most samples appear around approximately 5–15 ton/ha, 20–30 ton/ha, and 70–80 ton/ha.

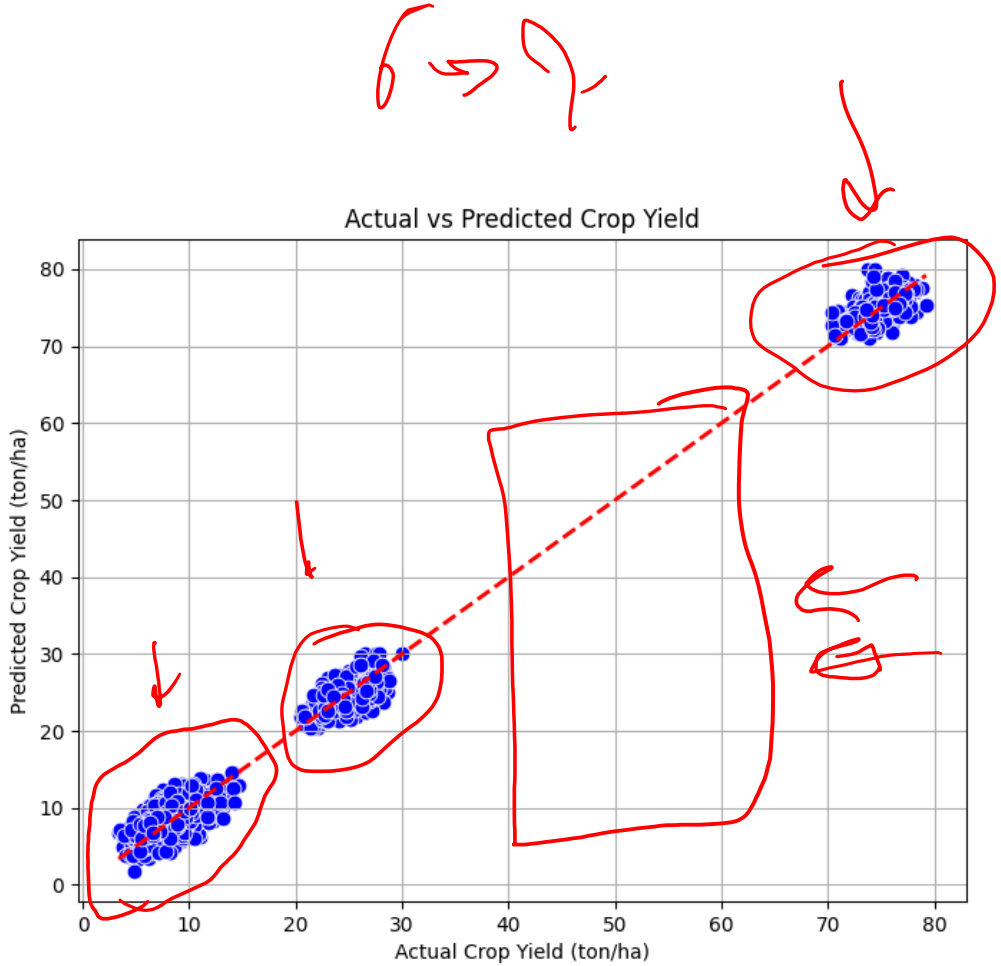


Figure 10: Actual vs Predicted Crop Yield

Reference:https://www.kaggle.com/datasets/miadul/smart-crop-recommendation-dataset/code

RTL Design

Decision Tree Rules: max depth = 8

```
1-  |--- Crop_Type <= 3.50
2-  |   |--- Crop_Type <= 1.50
3-  |   |   |--- Crop_Type <= 0.50
4-  |   |   |   |--- Fertilizer_Used <= 241.72
5-  |   |   |   |   |--- Rainfall <= 1684.32
6-  |   |   |   |   |   |--- Fertilizer_Used <= 108.78
7-  |   |   |   |   |   |   |--- Wind_Speed <= 3.80
8-  |   |   |   |   |   |   |   |--- Rainfall <= 438.85
9-  |   |   |   |   |   |   |   |   |--- value: [3.82]
.   |
.   |
.   |
.   |
761-| | | | | | | | |--- value: [11.39]
762-| | | | | | | |--- P >  60.50
763-| | | | | | | | |--- value: [12.35]
```

- Too many rules in this high-quality dataset:
  - Leading to inefficient RTL design coding.
- Solutions:
  1. Lookup Tables (LUTs) as Memory in FPGA
  2. Generate the nested if-else statements in HDL

Lookup Tables (LUTs) as Memory in FPGA

What is a LUT?

- Definition:
  - A Lookup Table (LUT) stores precomputed values for fast retrieval instead of real-time calculation.
- Used for logic functions or ROM/constant memory in FPGAs.
- Generating a HEX File:
  - Precompute table values (e.g., squares, sine/cosine).
  - Save in hex format compatible with FPGA memory tools.
- Benefits:
  - Saves logic resources.
  - Fast access to precomputed values.
  - Simplifies repetitive computations.

```
Hex file:
0000
0001
0004
…
…
…
0009
```

x

y

```
reg [15:0] memory [0:15];
initial $readmemh("lut.hex", memory);
assign data = memory[addr];
```

Generate the nested if-else statements in HDL

real to bit

3.6 → 64

Concept: These rules can be directly mapped into nested if-else statements in HDL.

LE

main_m4.py

```python
# ------------------------------
# Generate Verilog
# ------------------------------
feature_names = list(X.columns)
verilog_code = tree_to_verilog_if_else(tree_model, feature_names)
with open(f"{model_dir}/decision_tree.v", "w") as f:
    f.write(verilog_code)
```

```verilog
if (LE(Crop_Type, 64'h400C000000000000)) begin
    if (LE(Crop_Type, 64'h3FF8000000000000)) begin
        if (LE(Crop_Type, 64'h3FE0000000000000)) begin
            if (LE(Fertilizer_Used, 64'h406E370A40000000)) begin
                if (LE(Rainfall, 64'h409A5147A0000000)) begin
                    if (LE(Fertilizer_Used, 64'h405B3199A0000000)) begin
                        if (LE(Wind_Speed, 64'h400E666660000000)) begin
                            if (LE(Rainfall, 64'h407B6D9990000000)) begin
                                yield_f_reg = 64'h400E851EB851EB86;
```

Automatically generated RTL decision tree algorithm

7.63 → 1500

→ try LUTs

RTL Design

Decision Tree Rules: max depth = 8

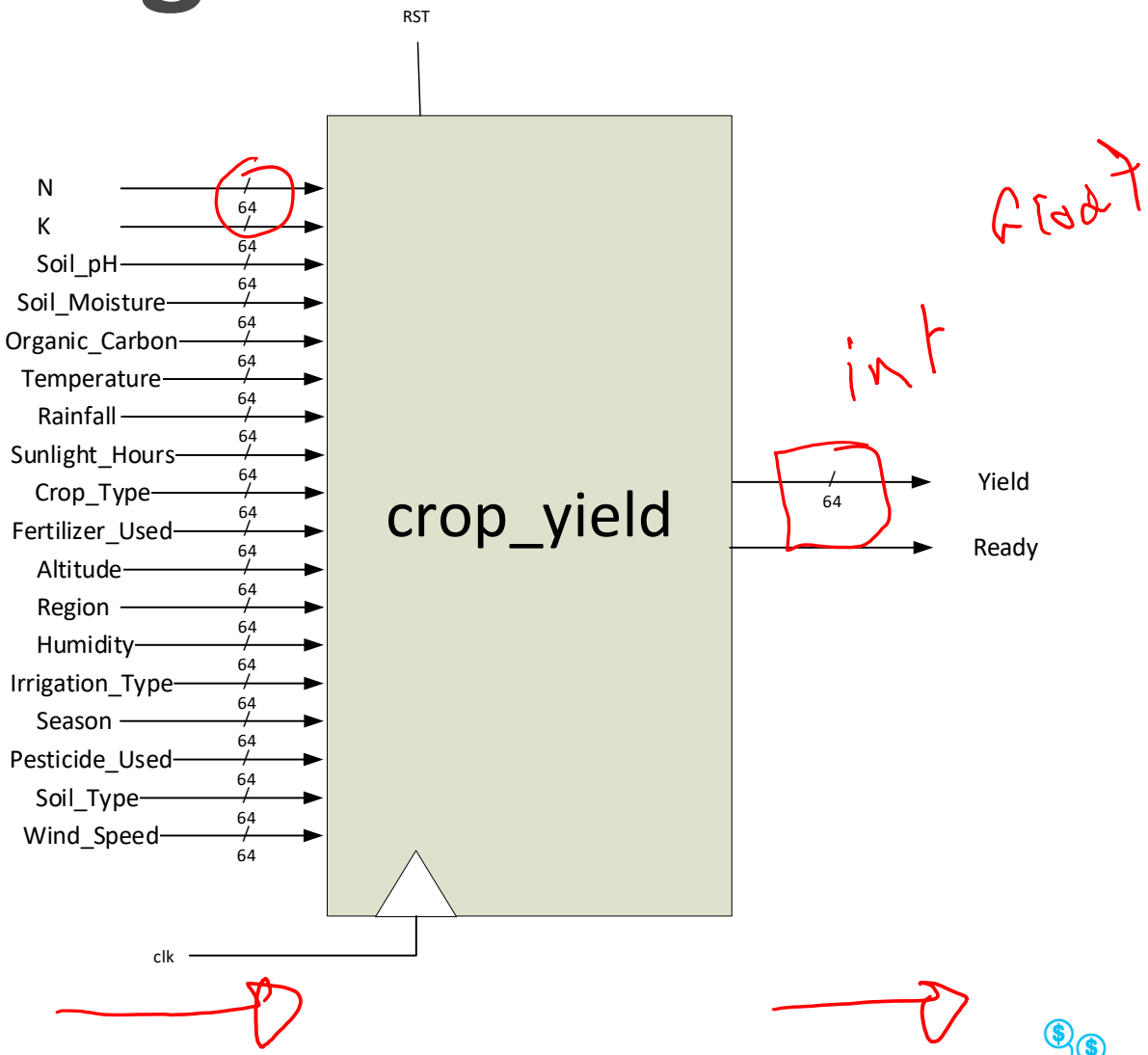
```
1-   |--- Crop_Type <= 3.50
2-   |    |--- Crop_Type <= 1.50
3-   |    |    |--- Crop_Type <= 0.50
4-   |    |    |    |--- Fertilizer_Used <= 241.72
5-   |    |    |    |    |--- Rainfall <= 1684.32
6-   |    |    |    |    |    |--- Fertilizer_Used <= 108.78
7-   |    |    |    |    |    |    |--- Wind_Speed <= 3.80
8-   |    |    |    |    |    |    |    |--- Rainfall <= 438.85
9-   |    |    |    |    |    |    |    |    |--- value: [3.82]
.    |
.    |
.    |
.    |
761- |  |  |  |  |  |  |  |  |--- value: [11.39]
762- |  |  |  |  |  |  |  |--- P >  60.50
763- |  |  |  |  |  |  |  |  |--- value: [12.35]
```

main.py

```
if (LE(Crop_Type, 64'h400C000000000000)) begin
    if (LE(Crop_Type, 64'h3FF8000000000000)) begin
        if (LE(Crop_Type, 64'h3FE0000000000000)) begin
            if (LE(Fertilizer_Used, 64'h406E370A40000000)) begin
                if (LE(Rainfall, 64'h409A5147A0000000)) begin
                    if (LE(Fertilizer_Used, 64'h405B3199A0000000)) begin
                        if (LE(Wind_Speed, 64'h400E666660000000)) begin
                            if (LE(Rainfall, 64'h407B6D9990000000)) begin
                                yield_f_reg = 64'h400E851EB851EB86;
                            end else begin  // Rainfall > 64'h407B6D9990000000
                                yield_f_reg = 64'h4019543B7543B755;
                            end
```

Final Design

- Note:
  - All inputs and outputs follow the IEEE 754 64-bit precision format except the ready signal



UVM Testbench

A scalable testbench to ensure design performs to specification

Conventual testbench results

The synthesizable design successfully predicts the yield with high accuracy

- As shown in the figure, 93 out of 106 are correct
- While only 13 out of the 5% tolerance, this is not a real problem since we can increase the depth of the tree to reach the required accuracy with just a single click.

```
# Row 96: Predicted Yield: 7.936842, Actual Yield: 7.418900
#   -> Correct (within 5%)
#
# Row 97: Predicted Yield: 9.315455, Actual Yield: 11.271300
#   -> Error (outside 5% tolerance)

# Simulation Summary:
#   Correct predictions: 93
#   Errors:              13
```

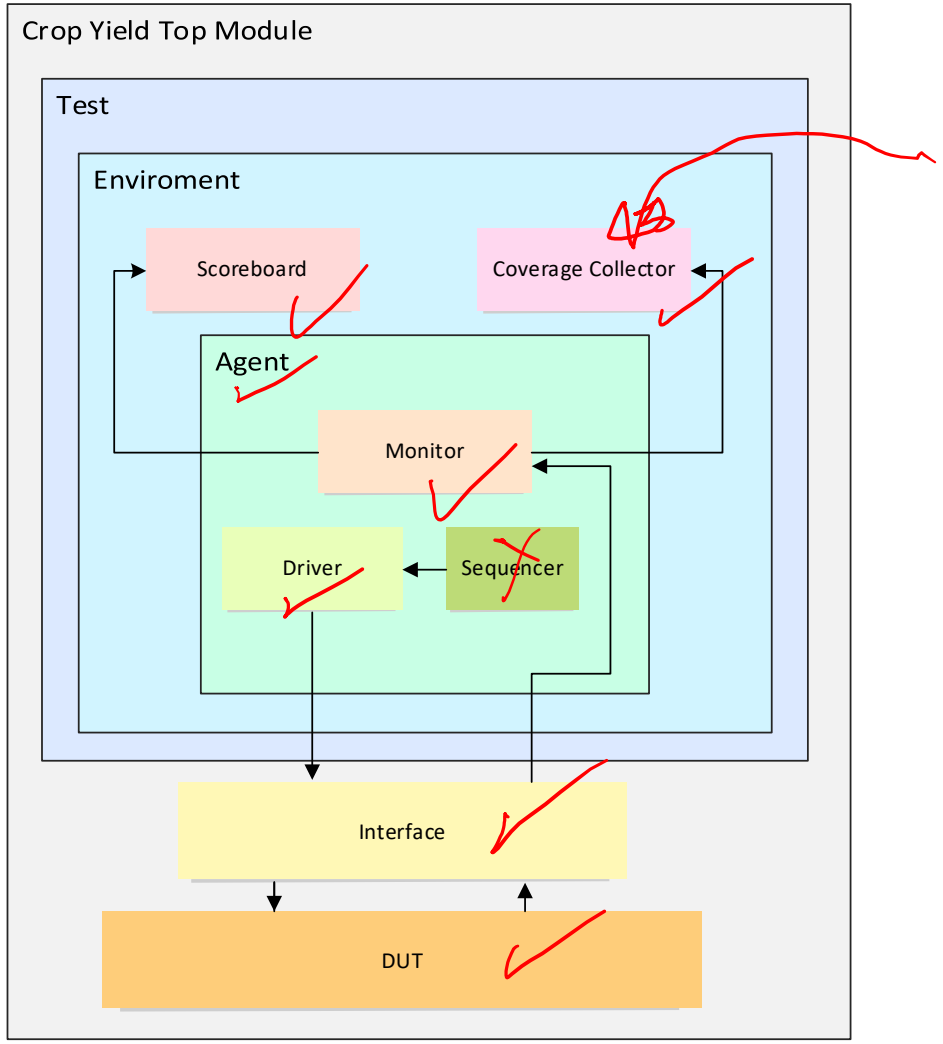
106

100%

1% -> 2%.

UVM Testbench

Crop Yield Top Module



UVM Scoreboard Comparison Approach

- The UVM Scoreboard is responsible for validating the DUT output against the reference design results.
- It receives transaction data from the monitor through an analysis export and FIFO.
- The scoreboard also reads reference outputs from design-generated HEX files:
  - `predicted_yield.hex`
  - `actual_yield.hex`

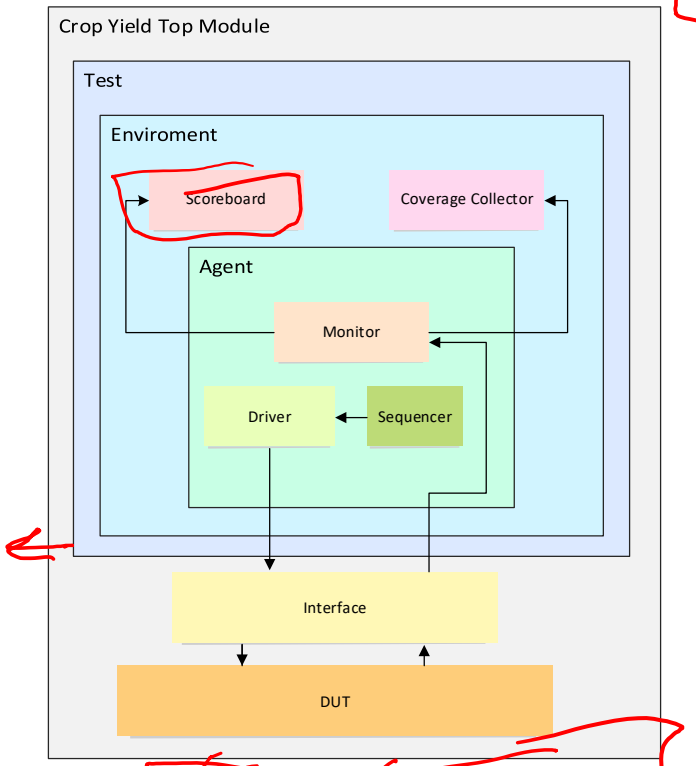
Comparison Flow

1. Monitor sends `cyp_seq_item` transactions to the scoreboard.
2. Transactions are stored in an analysis FIFO.
3. Scoreboard extracts the DUT output (Yield_Int).
4. Reference values are loaded from the HEX files into arrays:
   1. `predicted_yield_mem`
   2. `actual_yield_mem`

`actual_yield` Hex file:
0x4026d70a3d70a3d7
0x403730a3d70a3d71
…
0x401fc28f5c28f5c3

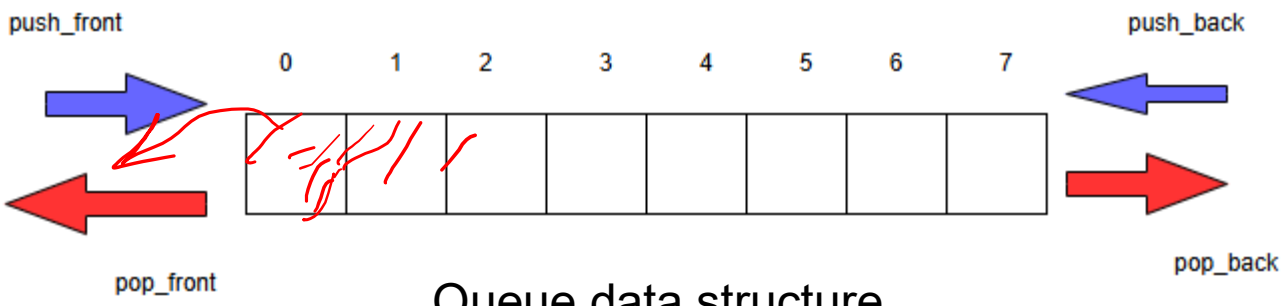
`predicted_yield` Hex file:
0x40276e147ae147ae
0x40382923c00b8ece
…
0x401eb645a1cac083

Hex file:
11.42
23.19
…
7.94



Scoreboard Comparison Approach

4. Reference values are loaded from the HEX files into arrays:
   1. `predicted_yield_mem`
   2. `actual_yield_mem`



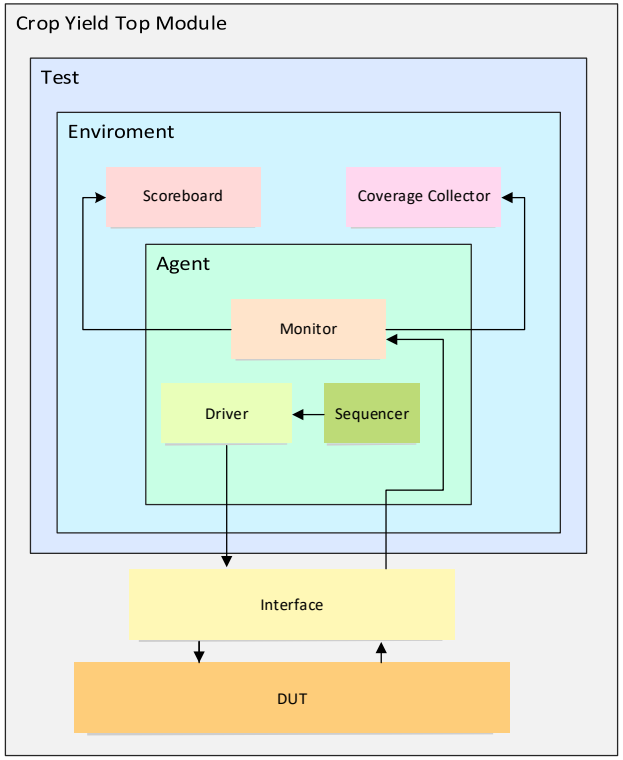
Queue data structure

```
logic [63:0] predicted_yield_mem[$];
logic [63:0] actual_yield_mem[$];
```

`actual_yield` Hex file:
0x4026d70a3d70a3d7
0x403730a3d70a3d71
…
0x401fc28f5c28f5c3

`predicted_yield` Hex file:
0x40276e147ae147ae
0x40382923c00b8ece
…
0x401eb645a1cac083

Hex file:
11.42
23.19
…
7.94



UVM testbench results

```
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100020: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(116) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Simulation Ends
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(117) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Correct Predicted: 10000
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(118) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Correct Actual:    9250
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(119) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Total Correct:     19250
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(120) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Error Predicted:   1
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(121) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Error Actual:      751
# UVM_INFO ./top/test/enviroment/scoreboard/cyp_scoreboard.svh(122) @ 100020: uvm_test_top.env.cyp_sb [REPORT] Total Errors:      752
```

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    15
# UVM_WARNING :    0
# UVM_ERROR :    1
# UVM_FATAL :    0
# ** Report counts by id
# [MISMATCH_PRED]     1
# [Questa UVM]     2
# [REPORT]     7
# [RNTST]     1
# [TEST_DONE]     1
# [run_phase]     4
```

Predic, RTL

1%.

Actual, RTL

20%.

```
[REPORT] Simulation Ends
[REPORT] Correct Predicted: 10000
[REPORT] Correct Actual:    9250
[REPORT] Total Correct:     19250
[REPORT] Error Predicted:   1
[REPORT] Error Actual:      751
[REPORT] Total Errors:      752
```

UVM Code Coverage Report

90%

Local Instance Coverage Details (81.63%)

Coverage Type ↑	Bins	Hits	Misses	Coverage
Branches	994	991	3	99.69%
Conditions	2	1	1	50%
Expressions	2	2	0	100%
Statements	508	505	3	99.4%
Toggles	2694	1591	1103	59.05%

Questa®
ADVANCED VERIFICATION
Questa Sim 2023.3
Copyright Mentor Graphics Corporation 1994-2023
Mentor®
A Siemens Business

SIEMENS

FPGA Flow

High-Level Design

Python code defines the intended behavior of the digital system

Register-Transfer Level

RTL describes the hardware implementation with finite state machines and datapath

Verification Environment

UVM testbench generates stimuli and monitors RTL output

Functional Verification

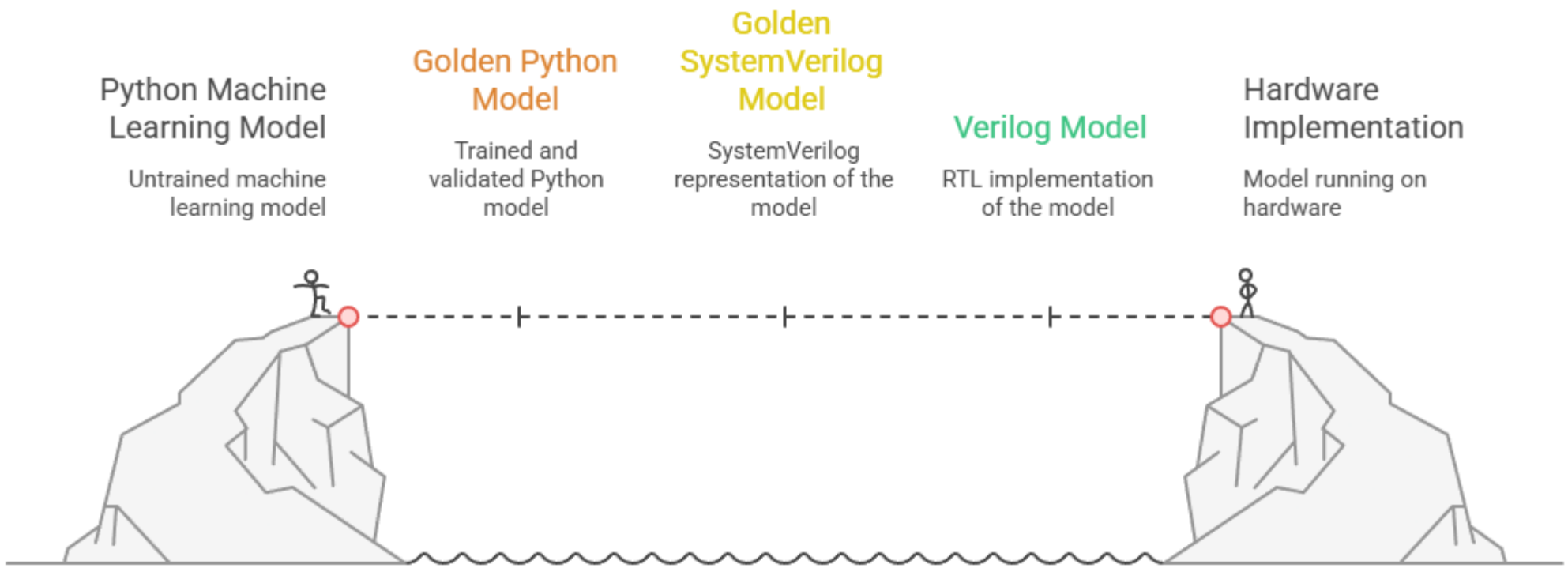
Scoreboard compares RTL output against Python golden model

Hardware Deployment

Verified RTL is synthesized and deployed onto an FPGA

Conclusion

- **RTL Status:** Fully synthesizable and ready for hardware deployment.
  Next Step: Proceed with the UVM functional verification environment and hardware deployment.



Notes

- All machine learning, design, and verification code is uploaded to GitHub.
- All presentation files are also on GitHub.
- The project uses a virtual Python environment, so users can run it with just two commands: make env and make run. 2
- The same simple approach is used for the FPGA workflow. With one click, the whole project can run on both Linux and Windows.
- **GitHub link:** https://github.com/H-Electro/ERI_SVM_FPGA_PROJECT

run.bat

run.sh

Thank you

Any Questions?